



Departamento de Informática e Matemática Aplicada – DIMAp

Serverless Computing

Prof. Nélio Cacho

DIM0614 – Programação Distribuída

Hora de silenciar o celular...

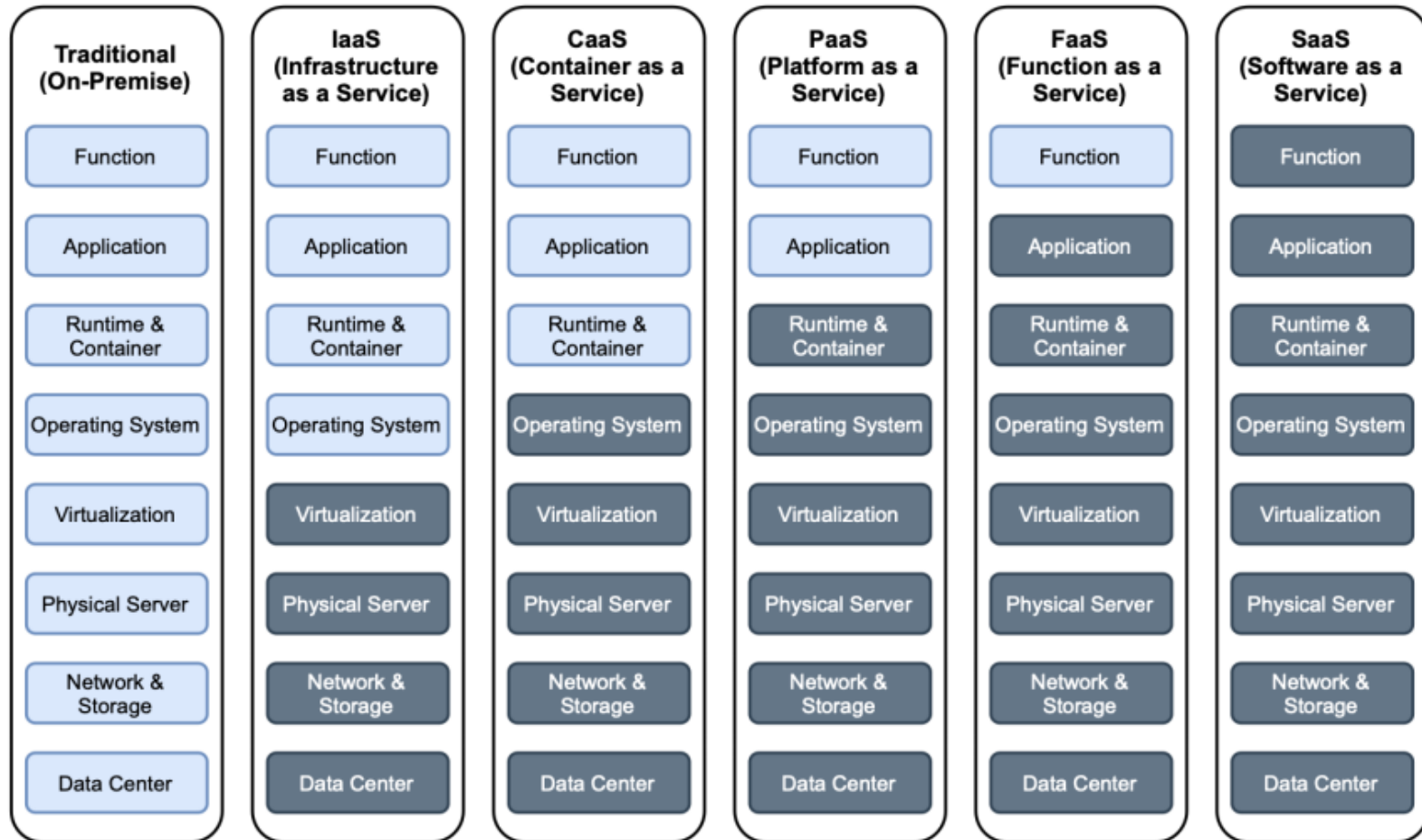


- Manter o telefone celular sempre desligado/silencioso quando estiver em sala de aula;
- Nunca atender o celular na sala de aula.

Conceitos

- Arquitetura Serverless, é uma arquitetura de computação **orientada a eventos**. Sua principal proposta é permitir que as empresas de software criem e mantenham seus aplicativos web sem se preocupar com a infraestrutura em que esses aplicativos estão rodando.
- *Serverless*, traduzido literalmente do inglês, significa “sem servidor”. Isso **não é verdade**. A arquitetura Serverless ainda é baseada em servidores. Nela, porém, o desenvolvedor não precisa configurar ou se preocupar com a maioria dos aspectos da infraestrutura em que sua aplicação será rodada. Essas decisões dinâmicas de infraestrutura ficam ocultas para o desenvolvedor ou operador.

Cloud Computing Service Models



LEGEND

Managed by the consumer

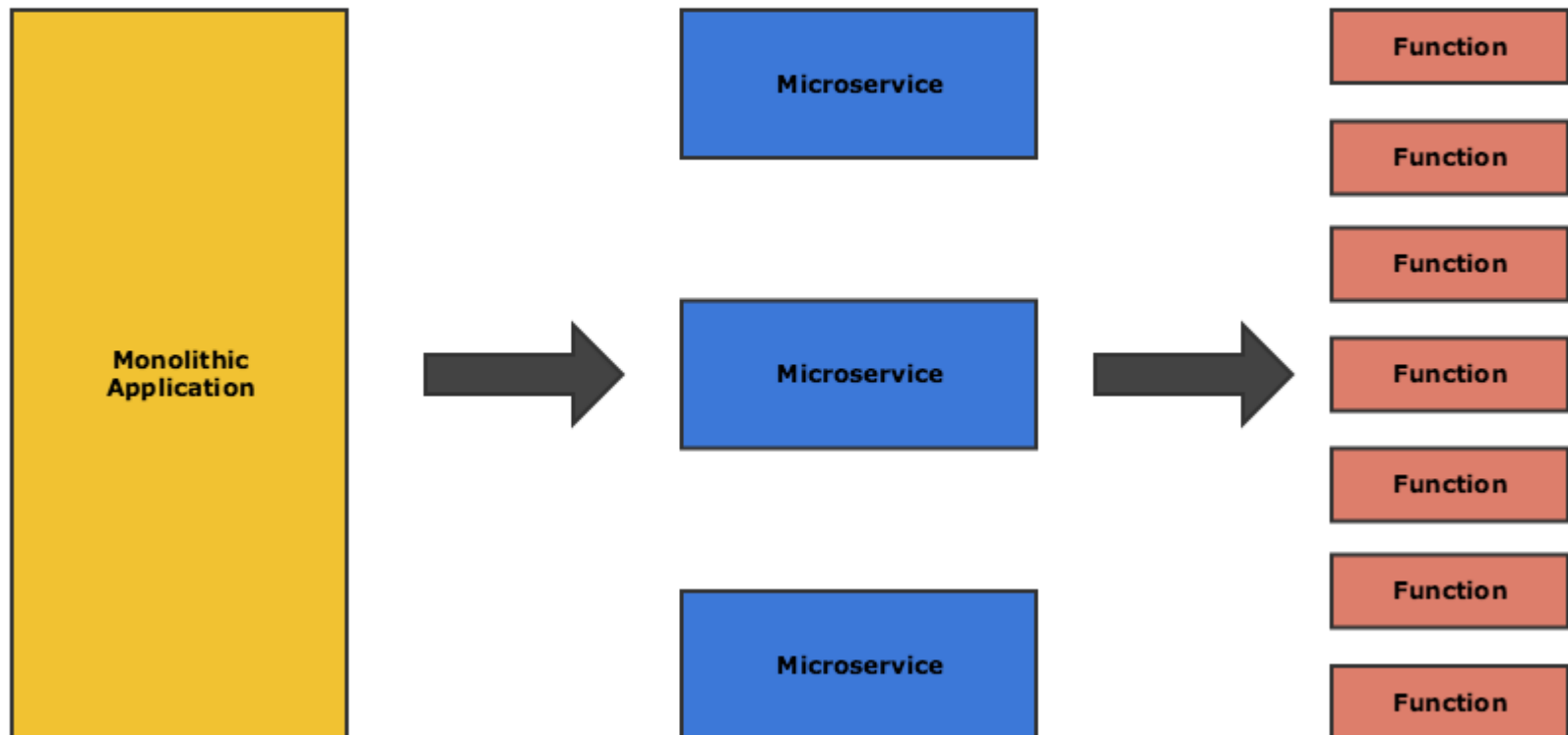
Managed by the cloud provider

Function as a Service (FaaS)

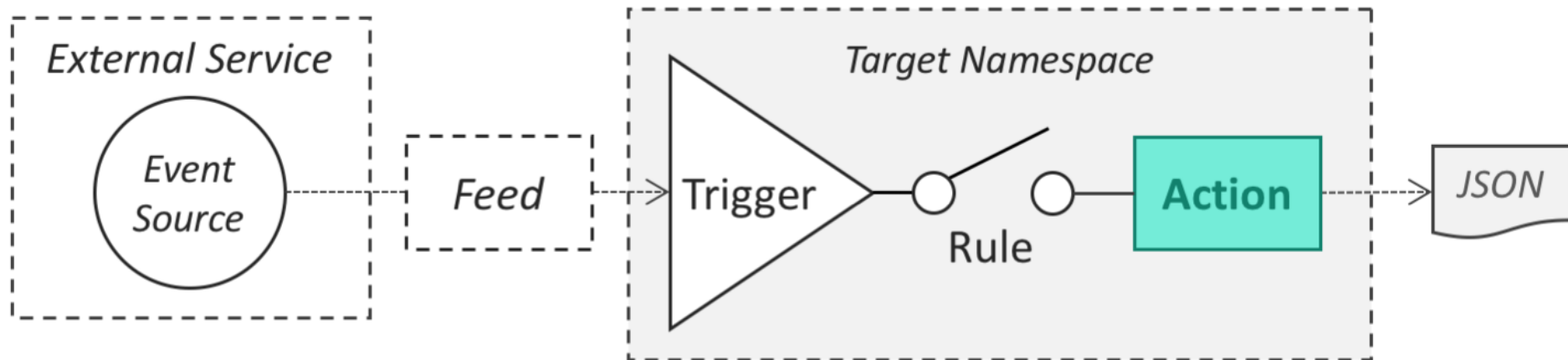
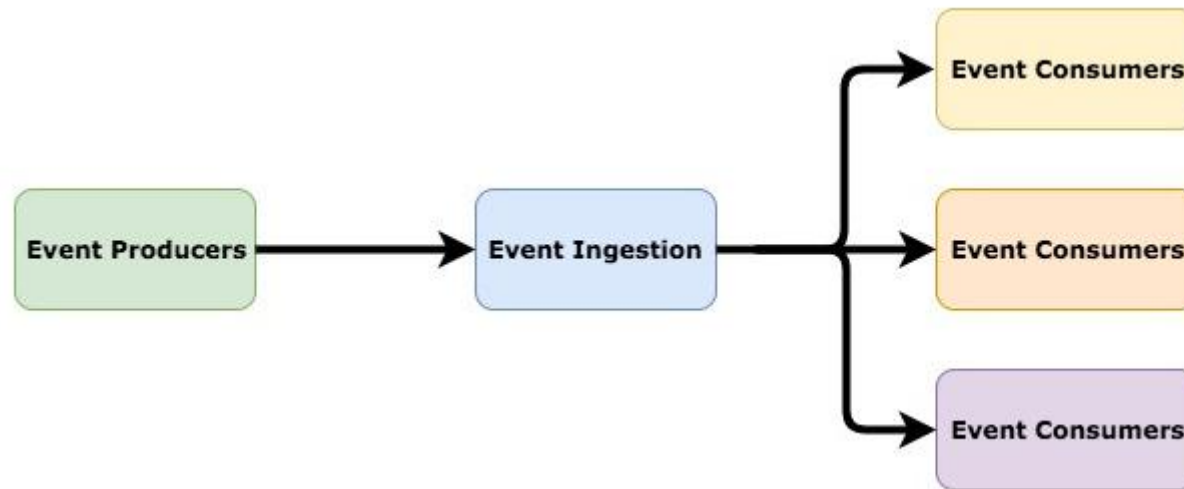
- Function as a Service (FaaS) é um serviço de computação em nuvem que busca abstrair a questão de servidor e máquinas onde rodam os seus programas.
- Em detalhes, ela deriva da filosofia Serverless que delega ao provedor de computação em nuvem (Cloud Provider) provisionar uma plataforma que permita ao cliente: desenvolver, rodar e gerenciar aplicações.
- Nestes moldes, reduz-se a complexidade de uma pessoa ter que montar e manter a infraestrutura, atual padrão de todo mercado corporativo. Montando uma aplicação com este padrão, implementamos a teoria Serverless e criamos uma arquitetura nativa de microsserviços.
- Provedores:
 - AWS: [AWS Lambda](#)
 - Microsoft Azure: [Azure Functions](#)
 - Google Cloud: [Cloud Functions](#)

Evolução

Monolithic vs Microservice vs FaaS



Como funciona ?

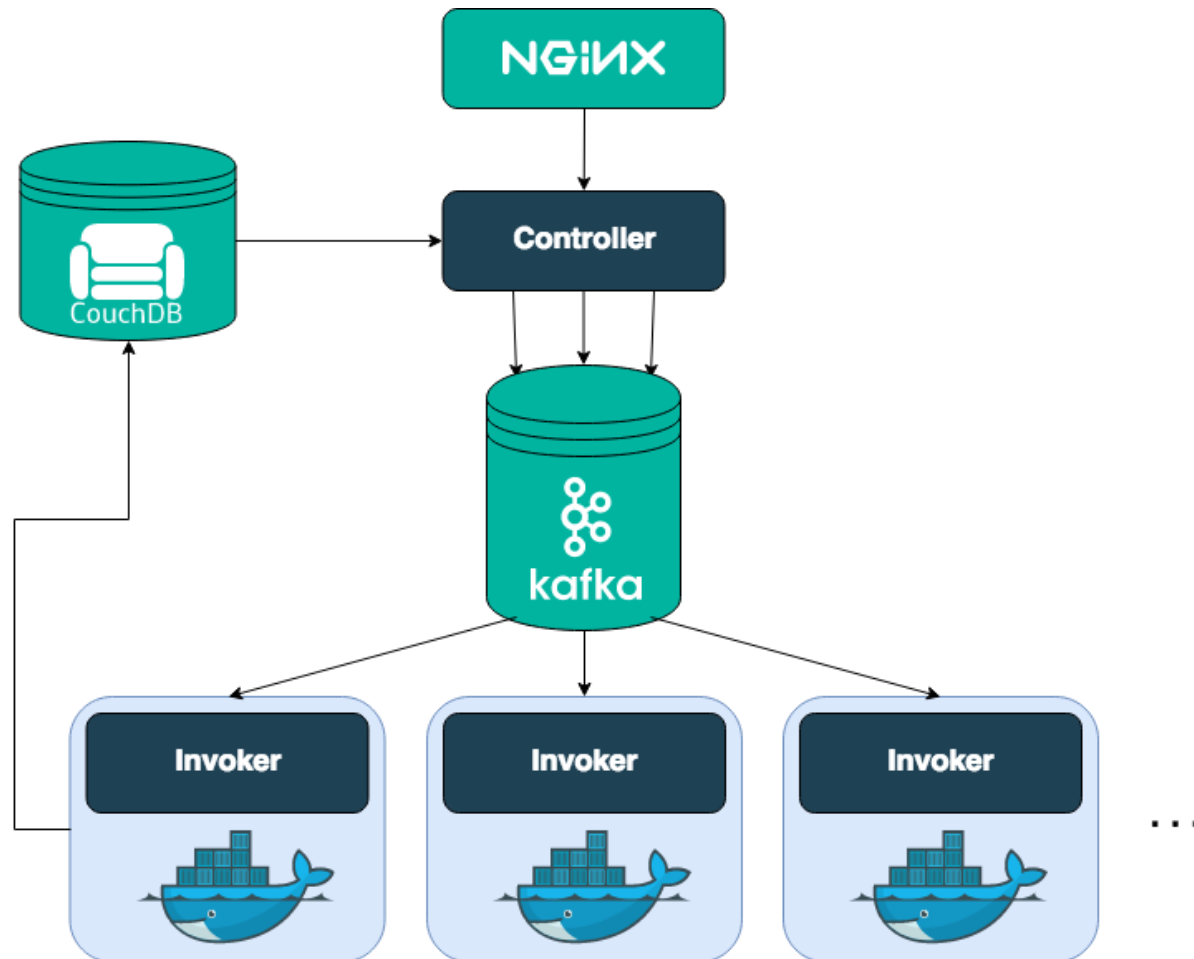


Como funciona ?

- A arquitetura *serverless* representa um modelo de hospedagem para funções que **não necessita configuração do servidor**, ou seja, todas dependências para que sua aplicação rode já estão instaladas de forma nativa.
- Essas funções podem ser "disparadas" de duas maneiras: Rotas HTTP assim como estamos acostumados em serviços REST comuns ou eventos disparados por outros serviços existentes.
- Geralmente a arquitetura *serverless* é utilizada para processar códigos assíncronos que não tem necessidade de **execução imediata** e que podem causar um processamento desnecessário na aplicação principal hospedada em servidores comuns.
- Importante frisar que toda função que utiliza esse modelo possui um limite de **tempo de execução** e é extremamente difícil de debugar ou testar, por isso, é importante que o serviço seja **extremamente simples** e tenha um objetivo bem definido com poucos efeitos colaterais.
- Além disso, esses serviços são **stateless** e, por isso, não devem guardar nenhum estado entre cada execução pois cada nova execução pode reconstruir o servidor do total zero sem resquícios de uma execução anterior.

Como funciona ?

- OpenWhisk Architecture



Casos de Uso

- **Micro-serviços:** Quando criamos aplicações através de micro-serviços podemos mover alguns dos serviços que escalam periodicamente para funções *serverless*;
- **Picos de acesso:** Imagine que sua aplicação receba picos de acesso durante períodos isolados, mover o serviço que processa essas operações para essa arquitetura evita um escalonamento desnecessário do servidor e redução de custos;
- **Serviço com pouco tráfego:** Outra utilidade é movermos serviços que não possuem tráfego constante para essa arquitetura já que, enquanto não executados, os serviços não são cobrados;
- **Diminuir *time-to-market*:** Caso você quera acelerar a velocidade de publicação de um projeto em fase inicial, a arquitetura *serverless* ajuda por fornecer uma estrutura de *deploy* simples com custo baixo;

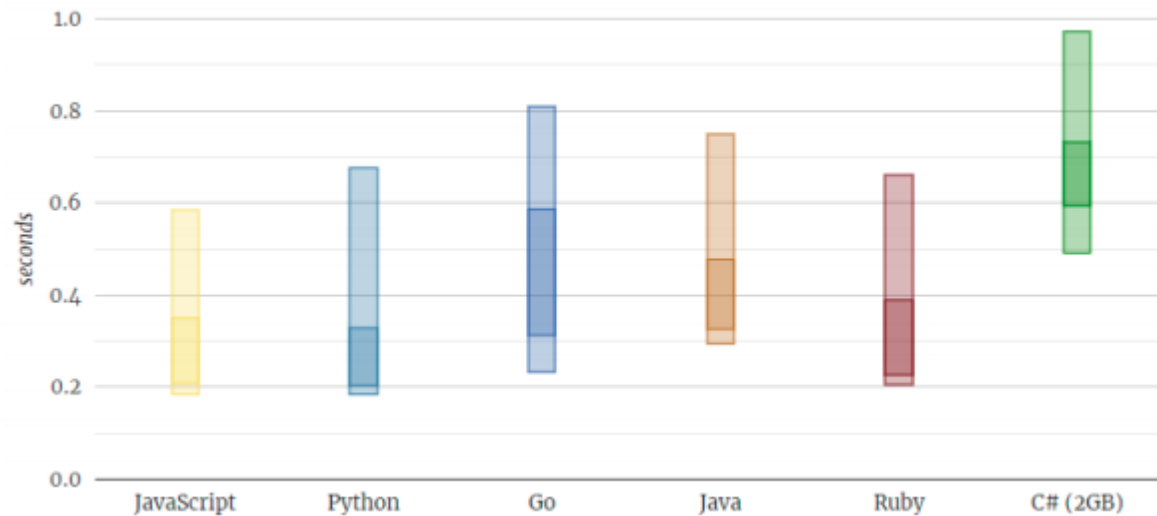
Vantagens do *serverless*

- Não precisamos **configurar o servidor** (Node, NGINX, Apache, etc...);
- **Extremamente barato:** U\$ 0,20/milhão de requisições na AWS (Fora tempo de computação caso seu serviço ocupe muito tempo);
- **Escalabilidade automática:** Não importa se tivermos uma ou 1 milhão de requisições, os servidores são auto-escaláveis e tudo isso é gerenciado pela provedora;
- **Deploy muito simples:** Subir novo código? Basta encapsular todo o código em um ZIP e subir pro servidor, o resto deixa com ele. Existem ferramentas que automatizam ainda mais isso;

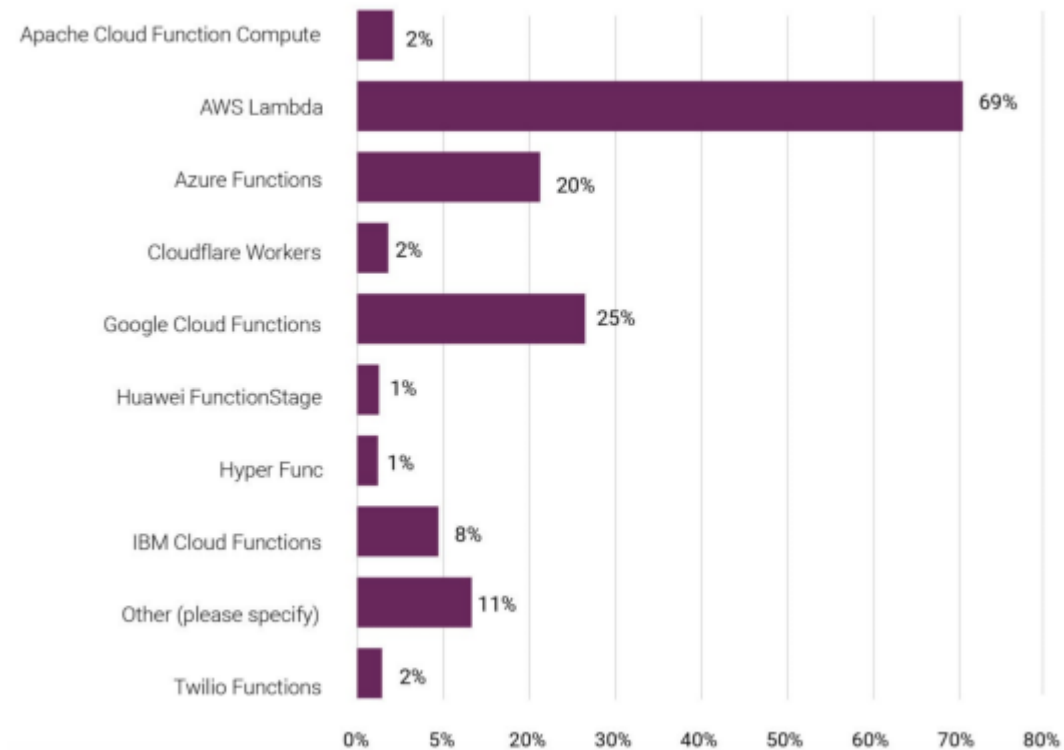
Desvantagens do *serverless*

- Execuções espaçadas criam o servidor do zero, realizando cold-starts da aplicação que podem ocasionar perda de performance momentânea;
- As funções executadas jamais podem passar de 300 segundos de execução, e se chegar perto disso o custo começa a aumentar bastante;
 - AWS Lambda—5 minutos
 - Azure Functions—10 minutos
 - Google Functions—9 minutos
- O debugging e teste das funções serverless é extremamente difícil, ainda mais quando essas dependem de eventos que ocorrem em outros serviços;
- Utilizado da forma errada o custo é maior que servidores cloud comuns já que serviços serverless não são criados para aplicações com execuções constantes;

Tempo estimado de Cold-Start



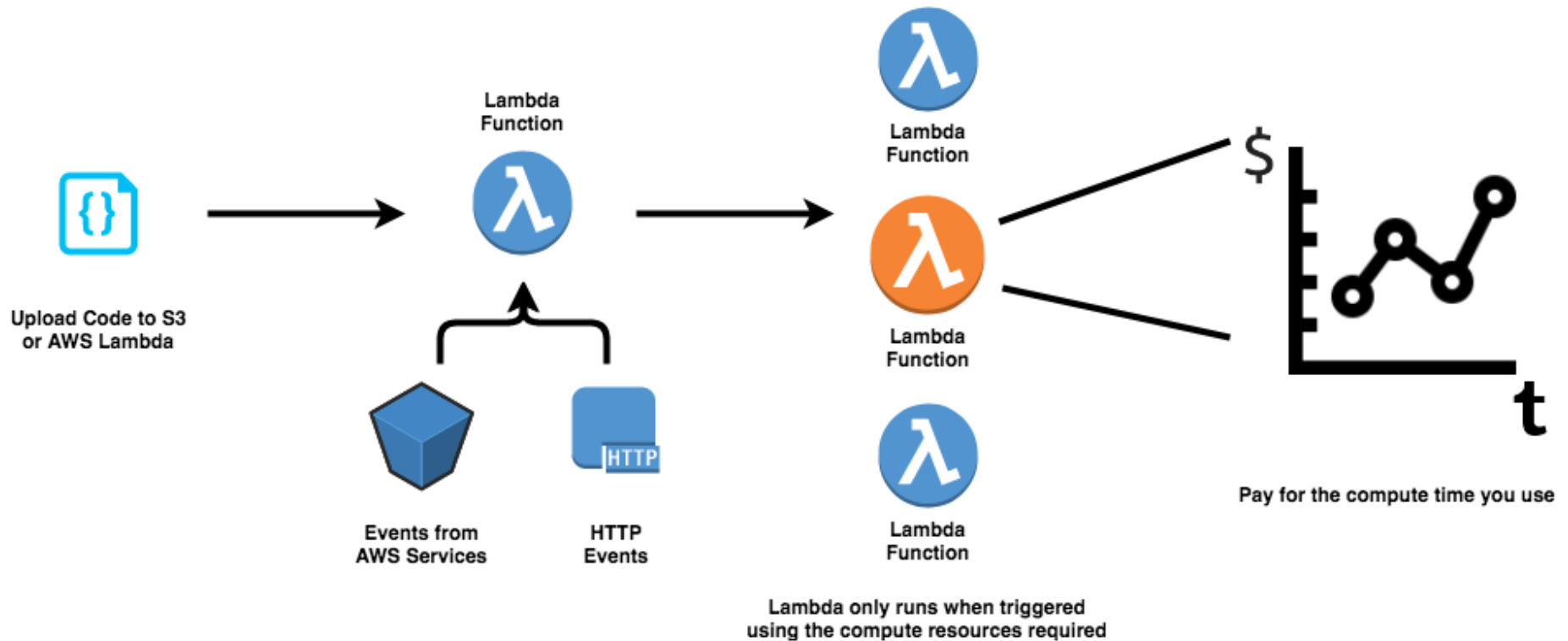
Utilização de Serverless



AWS Lambda

- O AWS Lambda permite que você execute códigos sem provisionar ou gerenciar servidores. Você paga apenas pelo tempo de computação que utilizar. Não haverá cobranças quando o seu código não estiver em execução.
- Linguagens suportadas:
 - Node.js
 - C#
 - Java
 - Python
 - Go

Como Funciona ?



Cenários

PROCESSAMENTO DE ARQUIVOS EM TEMPO REAL



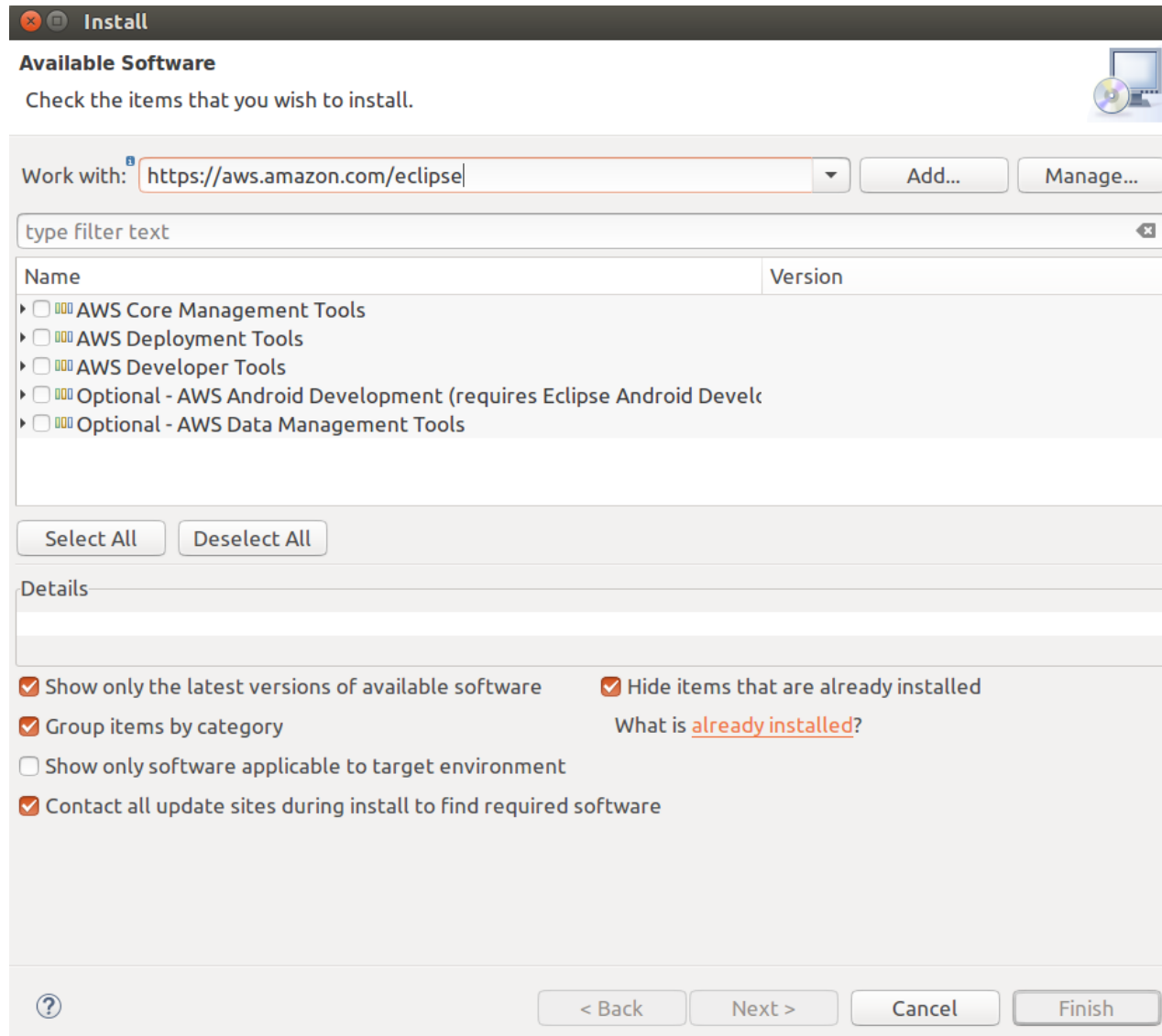
PROCESSAMENTO DE STREAM EM TEMPO REAL



EXTRAIR, TRANSFORMAR, CARREGAR



Criando Função em Java



Criando Função em Java



New AWS Lambda Maven Project

Create a new AWS Lambda Java project

Create a new AWS Lambda Java project in the workspace

Project name: demo

Maven configuration

Group ID: com.amazonaws.lambda

Artifact ID: demo

Version: 1.0.0

Package name: com.amazonaws.lambda.demo

Lambda Function Handler

Each Lambda function must specify a handler class which the service will use as the entry point to begin execution. [Learn more](#) about Lambda Java function handler.

Class Name: LambdaFunctionHandler

Input Type: S3 Event

An Amazon S3 trigger that retrieves metadata for the object that has been updated.

Preview:

```
package com.amazonaws.lambda.demo;

import com.amazonaws.services.lambda.runtime.Context;
import com.amazonaws.services.lambda.runtime.RequestHandler;
import com.amazonaws.services.lambda.runtime.events.S3Event;
import com.amazonaws.services.s3.AmazonS3;
import com.amazonaws.services.s3.AmazonS3ClientBuilder;
import com.amazonaws.services.s3.model.GetObjectRequest;
import com.amazonaws.services.s3.model.S3Object;

public class LambdaFunctionHandler implements RequestHandler<S3Event, String> {

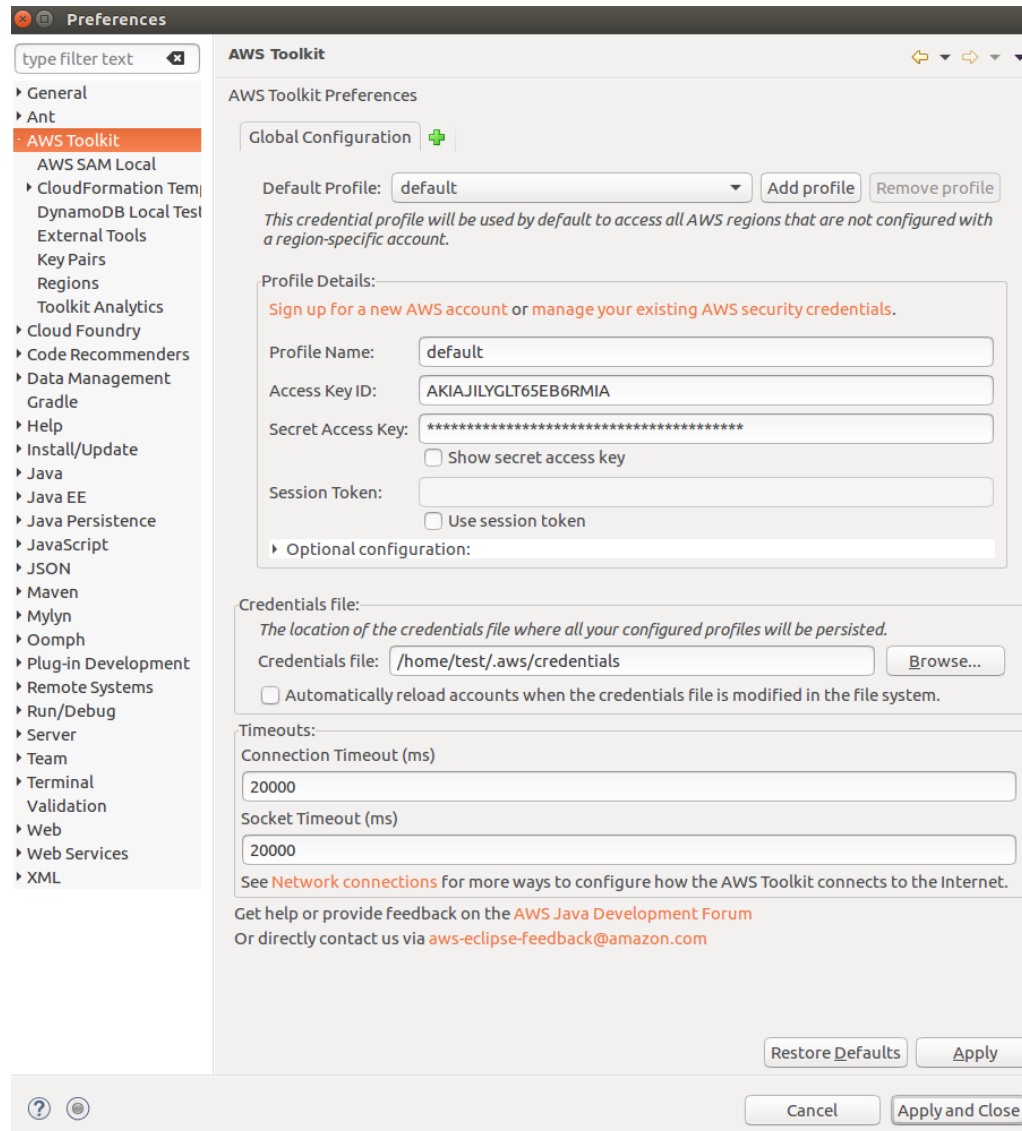
    private AmazonS3 s3 = AmazonS3ClientBuilder.standard().build();

    public LambdaFunctionHandler() {}

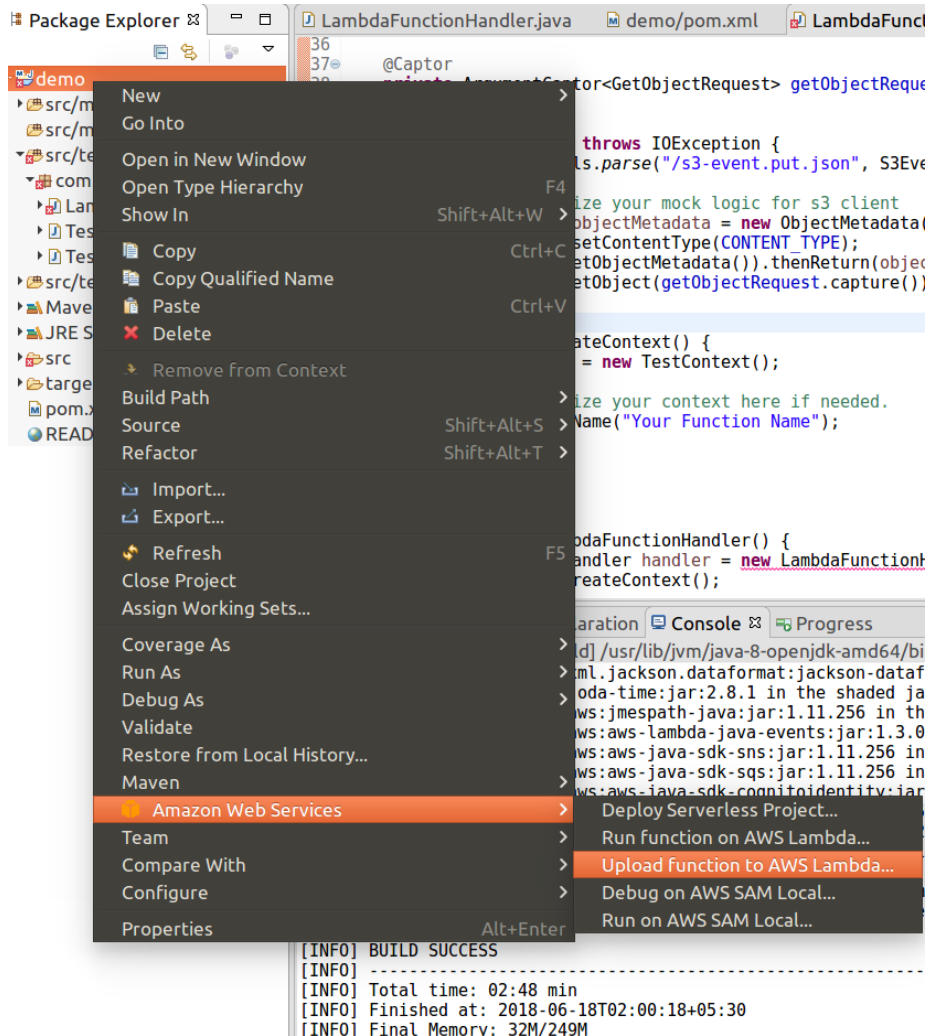
    // Test purpose only.
    LambdaFunctionHandler(AmazonS3 s3) {
        this.s3 = s3;
    }
}
```

Cancel Finish

Criando função em Java



Criando função em Java



Criando função em Java

  Upload Function to AWS Lambda

Select Target Lambda Function 

Choose the region and the target AWS Lambda function you want to create or update for your local lambda handler.

Select Lambda Handler and Target Region

Select the Handler:

Select the AWS Region:

Select or Create a Lambda Function:

☐ Choose an existing Lambda function:

☒ Create a new Lambda function:

Criando função em Java

Upload Function to AWS Lambda

Function Configuration

Configure this Lambda function and upload to AWS.

aws

Basic Settings

Name:

MyJavaFunction

Description:

The description for the function (optional)

Function Role

Select the IAM role that AWS Lambda can assume to execute the function on your behalf.
[Learn more about Lambda execution roles](#)

IAM Role:

lambda_basic_execution

Create

Function Versioning and Alias

You can publish a new immutable version and an alias to that version whenever you have a new revision of the Lambda function. [Learn more about Lambda function versioning and](#)

☐ Publish new version

☐ Provide an alias to this new version

☐ Choose an existing function alias:

Not Found

☒ Create a new function alias:

beta

S3 Bucket for Function Code

S3 Bucket:

lambda-function-bucket-eu-central-1-146179

Create

Upload Lambda zip file with encryption to protect data at rest by using Amazon S3 master-key or using AWS KMS master-key. [Learn more about Amazon S3 encryption](#)

☒ None

☐ Amazon S3 master-key

☐ AWS KMS master-key

KMS Key:

aws/lambda

Create

Advanced Settings

Memory (MB):

512

Timeout (s):

15

?

< Back

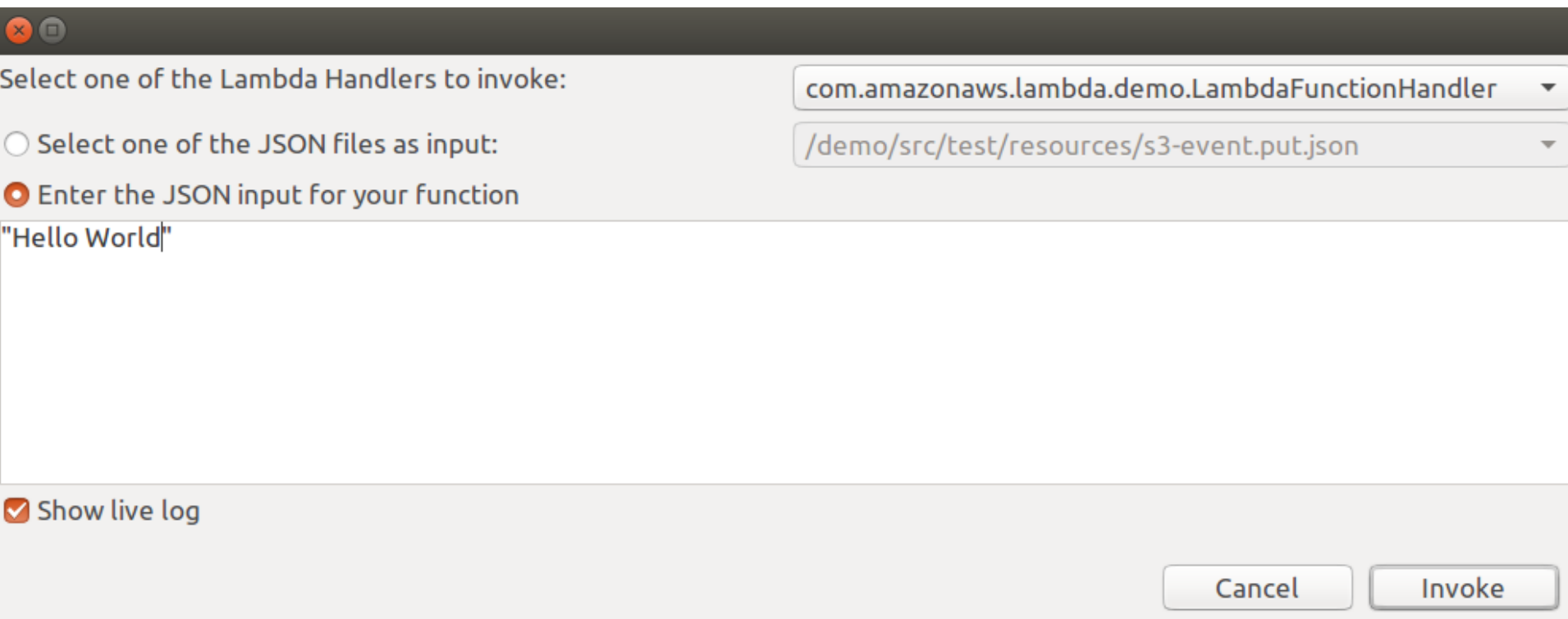
Next >

Cancel

Finish

Executando...

- Vá ao AWS Web Services e click em Run Function on AWS Lambda.



The screenshot shows the 'Run Function' dialog box in the AWS Lambda console. It has a dark grey title bar with standard window controls. The main area is light grey and contains three radio button options for selecting the input method. The first option, 'Select one of the Lambda Handlers to invoke:', is selected and has a dropdown menu showing 'com.amazonaws.lambda.demo.LambdaFunctionHandler'. The second option, 'Select one of the JSON files as input:', is unselected and has a dropdown menu showing '/demo/src/test/resources/s3-event.put.json'. The third option, 'Enter the JSON input for your function', is unselected. Below these options is a large text area containing the text '"Hello World"'. At the bottom left, there is a checked checkbox labeled 'Show live log'. At the bottom right, there are two buttons: 'Cancel' and 'Invoke'.

Select one of the Lambda Handlers to invoke:

☐ Select one of the JSON files as input:

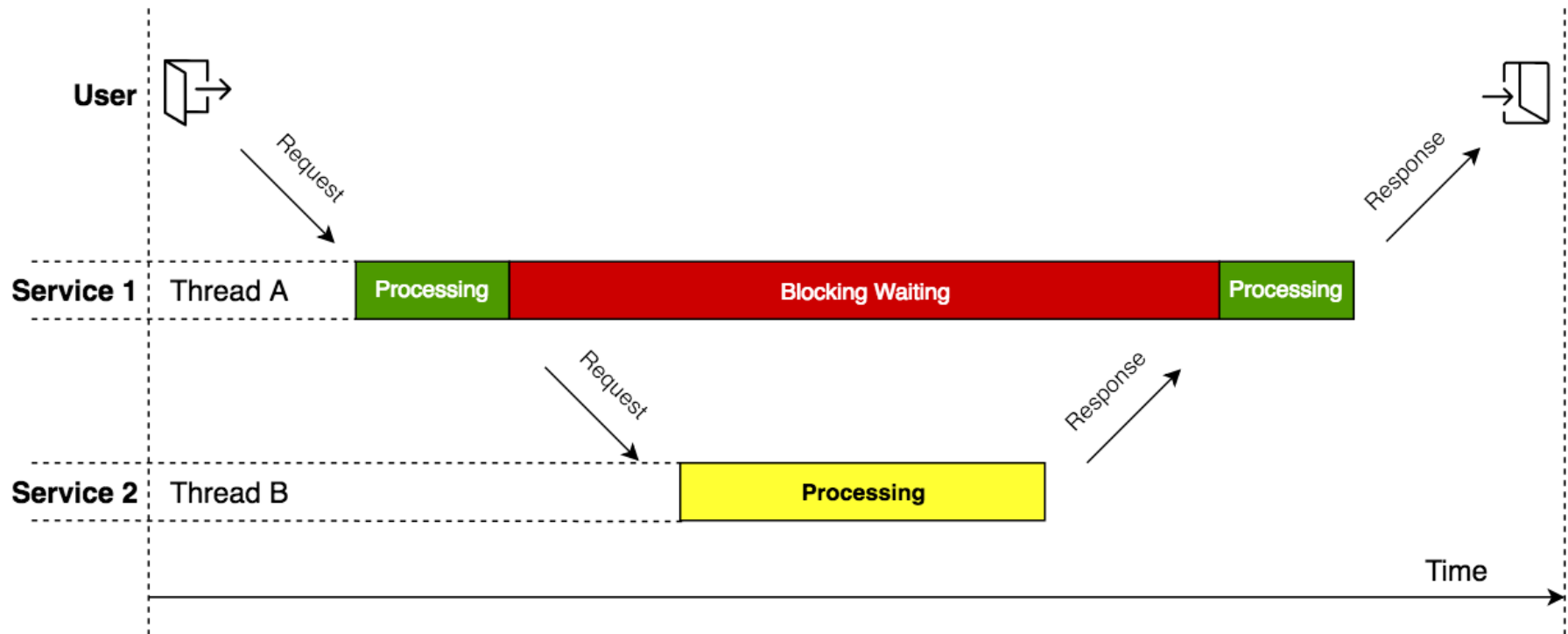
☒ Enter the JSON input for your function

"Hello World"

☒ Show live log

Cancel Invoke

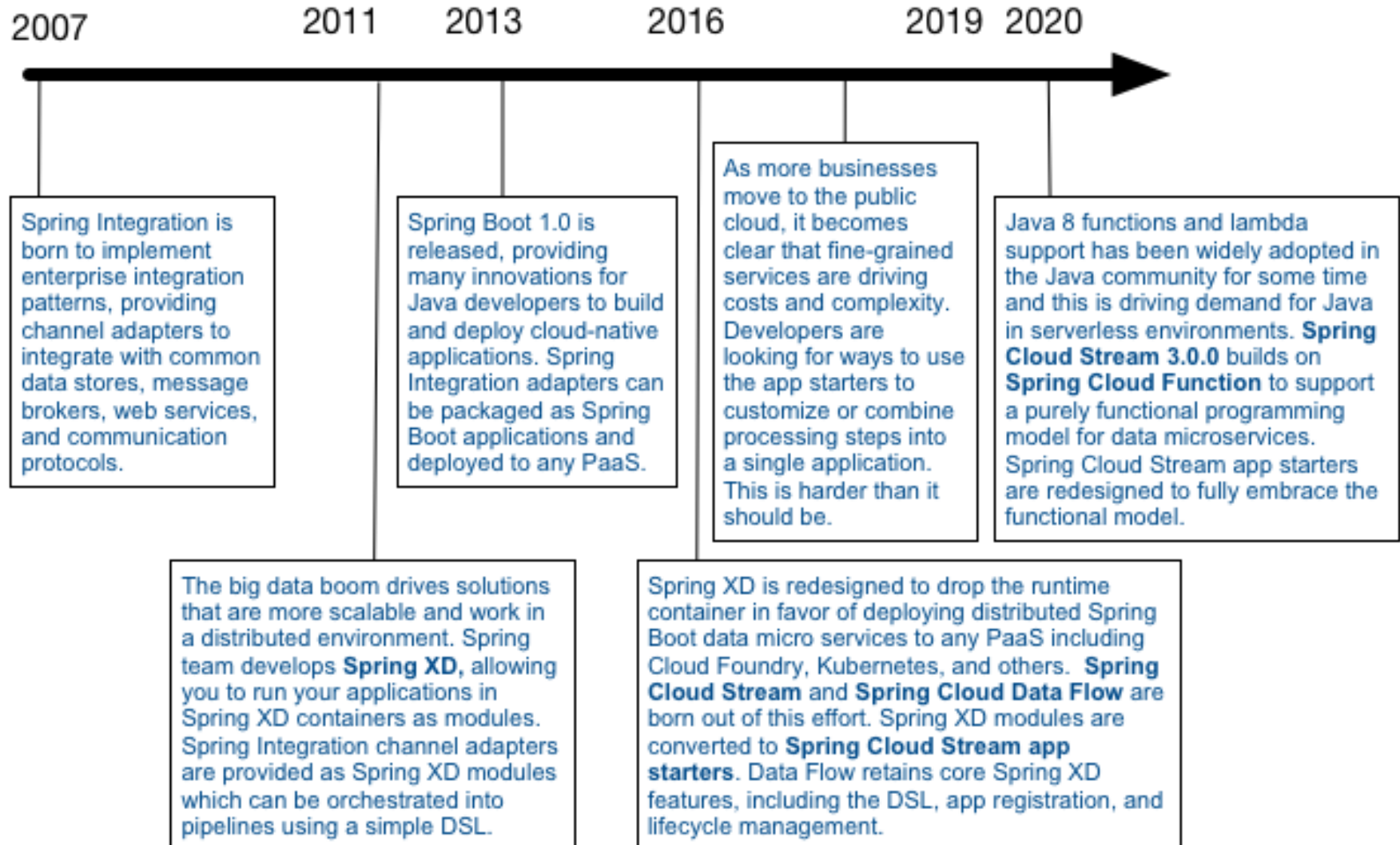
Serverless Computing em Java



Spring Cloud Stream

- O Spring Cloud Stream é um framework para construir microservices altamente escaláveis orientados a eventos, conectados a sistemas de mensagens compartilhados.
- O framework oferece um modelo de programação flexível construído em cima de idiomas e melhores práticas já estabelecidos e familiares do Spring, incluindo suporte para semântica pub/sub persistente, grupos de consumidores e partições com estado.

Spring Cloud Stream



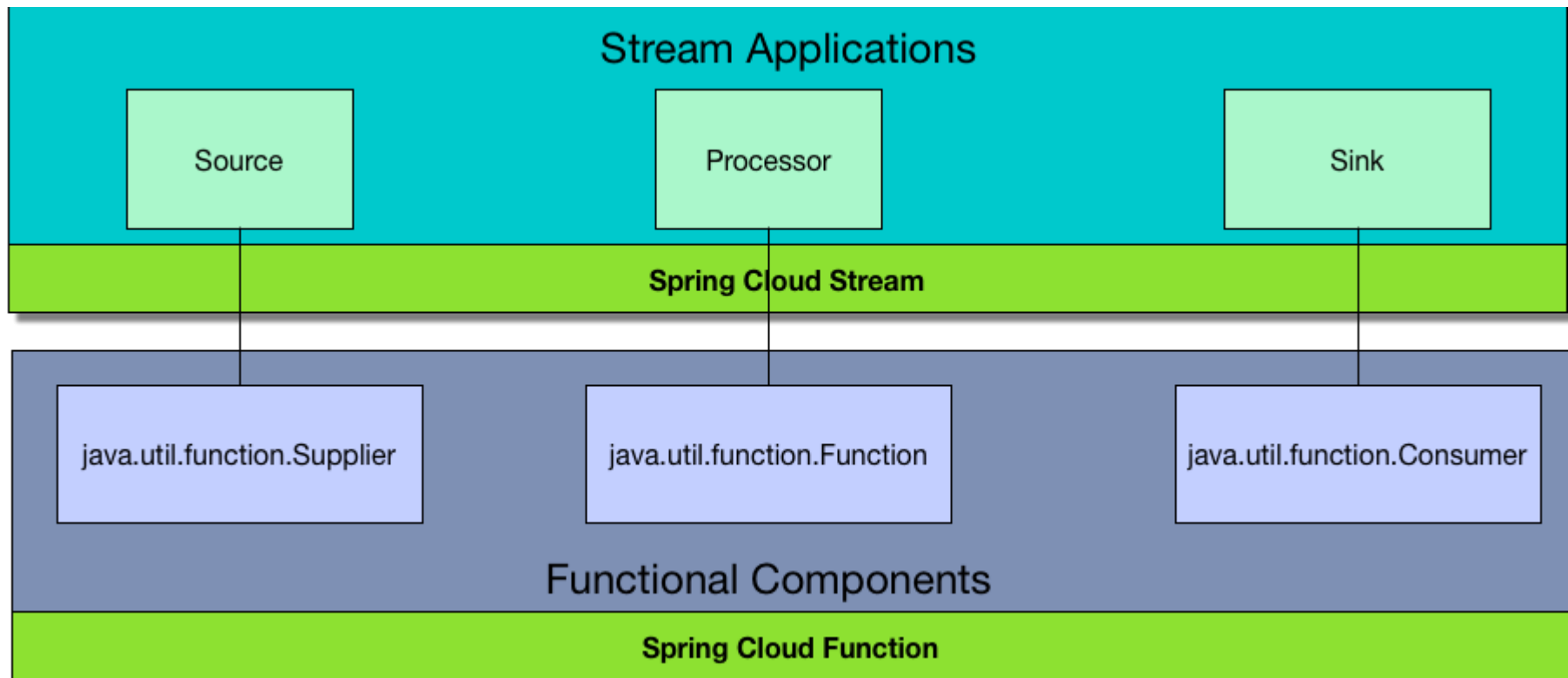
Spring Cloud Stream

- Na versão 3.x do Spring Cloud Stream, foi introduzido um modelo de programação funcional baseado no **Spring Cloud Function**.
- Essa abordagem é preferida em relação ao modelo antigo baseado em anotações (@EnableBinding, @StreamListener).
- O Spring Cloud Stream pode se conectar diretamente às entradas e saídas de um Bean de Função. Com esse modelo, as anotações antigas do Spring Cloud Stream e as interfaces Source, Processor e Sink não são mais necessárias.

Spring Cloud Stream com Stream Cloud Function

- Os componentes funcionais estão disponíveis para serem empacotados e implantados em uma ampla gama de casos de uso além do Spring Cloud Stream, especialmente em ambientes FaaS (Function as a Service).
- Os componentes funcionais são construídos com o Project Reactor de modo a permitir streaming reativo não bloqueante.

Spring Cloud Stream com Stream Cloud Function



Spring Cloud Function

- Promove a implementação da lógica de negócio por meio de funções.
- Desvincula o ciclo de vida do desenvolvimento da lógica de negócio de qualquer destino de execução específico, de modo que o mesmo código possa ser executado como um endpoint web, um processador de fluxo ou uma tarefa.

Spring Cloud Function

```
public interface Function<T, R> {  
    R apply(T t);  
}  
  
public interface Consumer<T> {  
    void accept(T t);  
}  
  
public interface Supplier<T> {  
    T get();  
}
```

- *Function*: recebe um valor, processa e retorna o valor.
- *Consumer*: usada para salvar o valor recebido e persistir
- *Supplier*: retorna algo sem receber parâmetro.

Criando uma Spring Cloud Function



New Spring Starter Project

Service URL:

Name:

☒ Use default location

Location:

Type: Packaging:

Java Version: Language:

Group:

Artifact:

Version:

Description:

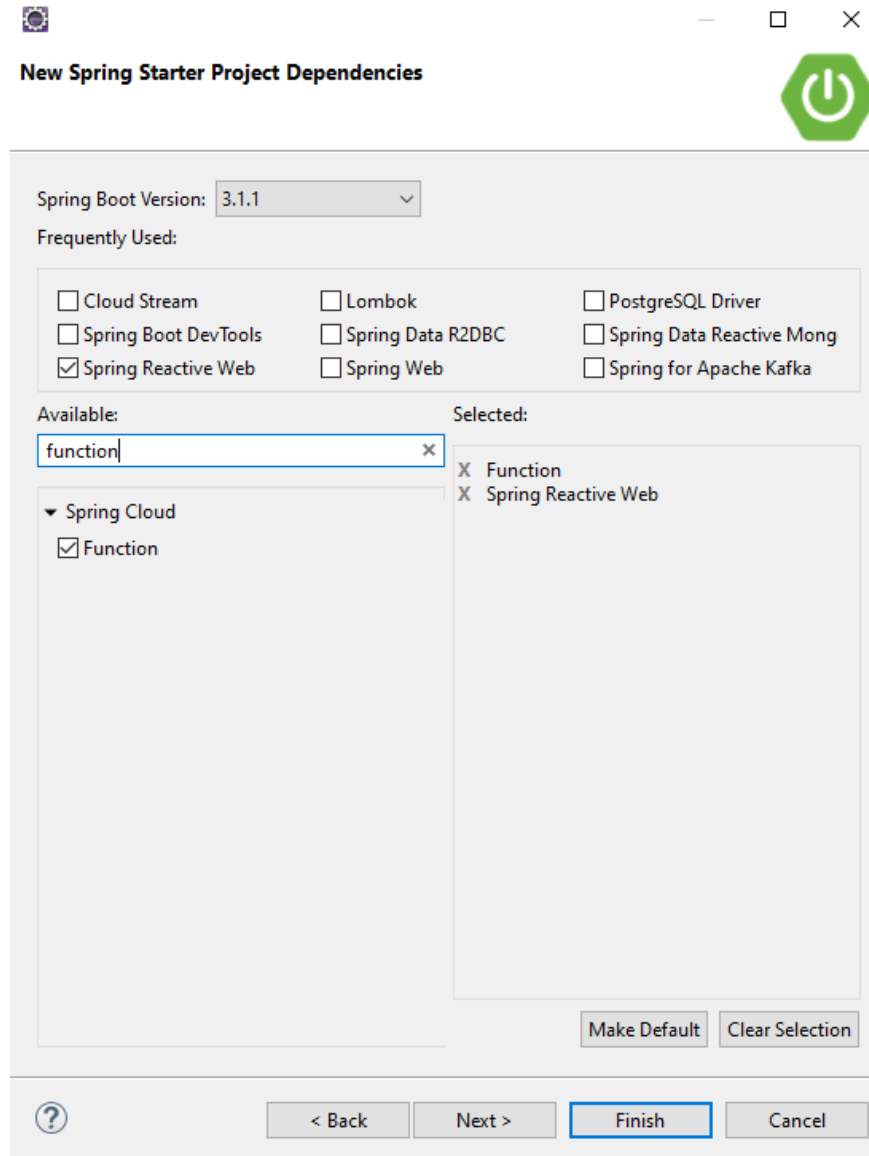
Package:

Working sets

☐ Add project to working sets

Working sets:

Criando uma Spring Cloud Function



The image shows a 'New Spring Starter Project Dependencies' dialog box. At the top, it has a title bar with a question mark icon, standard window controls (minimize, maximize, close), and a green power button icon. The main content area is divided into several sections. The 'Spring Boot Version' is set to '3.1.1'. Below it, the 'Frequently Used' section contains a grid of checkboxes for various dependencies: Cloud Stream, Lombok, PostgreSQL Driver, Spring Boot DevTools, Spring Data R2DBC, Spring Data Reactive Mong, Spring Reactive Web (checked), Spring Web, and Spring for Apache Kafka. The 'Available' section on the left has a search bar with 'function' entered and a list showing 'Spring Cloud' expanded with 'Function' checked. The 'Selected' section on the right shows 'Function' and 'Spring Reactive Web' with 'X' marks. At the bottom right of the main area are 'Make Default' and 'Clear Selection' buttons. The bottom of the dialog features a footer bar with a question mark icon, '< Back', 'Next >', 'Finish' (highlighted with a blue border), and 'Cancel' buttons.

New Spring Starter Project Dependencies

Spring Boot Version: 3.1.1

Frequently Used:

☐ Cloud Stream	☐ Lombok	☐ PostgreSQL Driver
☐ Spring Boot DevTools	☐ Spring Data R2DBC	☐ Spring Data Reactive Mong
☒ Spring Reactive Web	☐ Spring Web	☐ Spring for Apache Kafka

Available:

function

▼ Spring Cloud

- ☒ Function

Selected:

- X Function
- X Spring Reactive Web

Make Default Clear Selection

? < Back Next > Finish Cancel

Criando uma Spring Cloud Function

- Importar a dependência:

```
<dependency>  
  <groupId>org.springframework.cloud</groupId>  
  <artifactId>spring-cloud-starter-function-web</artifactId>  
</dependency>
```

As bibliotecas do Spring Cloud Function promovem e suportam a implementação da lógica de negócio por meio de funções.

Criando uma Spring Cloud Function

- O Spring Cloud Function é capaz de gerenciar funções definidas de diferentes maneiras, desde que sigam as interfaces padrão do Java: **Function**, **Supplier** e **Consumer**.
- O Spring Cloud Function suporta tanto código imperativo quanto reativo, permitindo que você implemente funções usando APIs reativas como Mono e Flux. Você também pode combinar as abordagens, se desejar.

Criando uma Spring Cloud Function

```
@Configuration
public class OrderFunction {
    @Bean
    public Function<String, String> gerarordemcompra(){
        return str -> {
            System.out.println("O valor recebido foi:"+str);
            return "Ordem de compra:" + ((String) str).toUpperCase();
        };
    }
}
```

Criando uma Spring Cloud Function

```
INFO 8100 --- [main] b.d.ufrn.CloudFunctionOrderApplication : Starting CloudFunctionOrderApplication
INFO 8100 --- [main] b.d.ufrn.CloudFunctionOrderApplication : No active profile set, falling back to
INFO 8100 --- [main] o.s.c.f.web.flux.FunctionHandlerMapping : FunctionCatalog: org.springframework.c
INFO 8100 --- [main] o.s.b.web.embedded.netty.NettyWebServer : Netty started on port 8080
INFO 8100 --- [main] b.d.ufrn.CloudFunctionOrderApplication : Started CloudFunctionOrderApplication
```

← → ↻ ⓘ localhost:8080/gerarordemcompra/ordem:nelio:01234

Ordem de compra:ORDEM:NELIO:01234